Table des matières

Cours 06 — Orchestrer et rattraper 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Le jour logique · Tenir le pipeline 2
Étape 2 — Le chargement d'un bloc, mis de côté · Tenir le pipeline 3
Étape 3 — Rattraper juin, jour par jour · Tenir le pipeline 4
Étape 4 — Prouver que le rattrapage égale le chargement d'un bloc · Tenir le pipeline 5
Étape 5 — Le rattrapage qui s'arrête · Tenir le pipeline 6
Étape 6 — Versionner · Tenir le pipeline 6
À retenir 7

Cours 06 — Orchestrer et rattraper

Cours 6/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
✓ ici · Tenir le pipeline, ce qui traverse les trois étapes : le code, l'orchestration, la qualité	✓	✓

Ce que vous allez faire

En exploitation, personne ne lance le pipeline à la main : un orchestrateur le lance chaque nuit, pour la journée qui vient de finir. Et quand il a été arrêté une semaine, il rattrape la semaine, jour par jour, dans l'ordre. Ce cours fait la part de ce travail qui ne demande pas d'orchestrateur : le jour logique, le rattrapage, et la preuve que rattraper un mois jour par jour donne la même `RAW` que le charger d'un bloc. Vous écrivez `rattraper.py`, qui appelle la porte d'entrée du cours 05 une fois par jour.

le mois chargé d'un bloc, mis de côté → la `RAW` effacée → `rattraper.py`, trente jours logiques → les deux `RAW` comparées dans les deux sens → le rattrapage arrêté au premier échec

© 2026 iAlexMG. Tous droits réservés.

Le code complet est dans `Codes_06_DE.md`, à ouvrir après avoir essayé.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour le Bloc-notes, Python et Git ; la seconde pour le notebook, qui s'ouvre à l'étape 4. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
python 01_de\pipeline.py 2025-06-30
REM Attendu : bronze : 1527 tickets rangés, puis raw : 75 fichiers au journal.
```

Ce cours part du pipeline du cours 05. S'il s'arrête, reprenez la fin du cours 05.

Étape 1 — Le jour logique · Tenir le pipeline

Le jour logique d'une exécution est la journée dont elle traite les livraisons, pas le jour où elle tourne. La nuit du 13 juin, l'orchestrateur lance le pipeline pour le jour logique du 12 ; rattrapé en octobre, le même travail porte toujours le jour logique du 12, et charge exactement ce qui était arrivé ce jour-là. C'est le rôle de l'argument de `pipeline.py` depuis le cours 05 : le jour jusqu'où le chargeur prend les fichiers.

Rattraper une période, c'est lancer le pipeline une fois par jour logique, du premier au dernier, dans l'ordre, et s'arrêter au premier échec : un jour manqué laisse un trou, et les jours suivants attendent qu'il soit réglé. Seule la `RAW` suit le jour logique ; le bronze, lui, se refait en entier à chaque exécution, comme chez Boréal. Airflow, l'orchestrateur de Boréal au même cours, ne tourne pas sous Windows sans WSL ni Docker : il reste à Boréal.

Pour remplir	**L'outil**
lire un jour	`date.fromisoformat("2025-06-01")` rend une date, qui s'additionne et se compare
le jour suivant	`jour + timedelta(days=1)`
redonner le jour en texte	`jour.isoformat()` rend `2025-06-01`, la forme que `pipeline.py` attend
lancer une exécution	`lancer(jour)`, la fonction de `pipeline.py`, qui rend son code de sortie

1.1 Créer le fichier, dans la première fenêtre.

```
notepad 01_de\rattraper.py
```

Accepter la création, puis écrire le code à partir de ce pseudocode, les trous ____ compris :

© 2026 iAlexMG. Tous droits réservés.

```
# 01_de/rattraper.py : rattraper une période jour par jour, dans l'ordre, jusqu'au premier échec
import sys
from datetime import date, timedelta

from pipeline import lancer

def jours_logiques(depuis, jusqua):
    # 1. chaque jour de la période, du premier au dernier, bornes comprises
    jour, fin, jours = date.fromisoformat(depuis), date.fromisoformat(jusqua), []
    while jour <= fin:
        jours.append(jour.isoformat())
        jour += ____
    return jours

def rattraper(depuis, jusqua):
    for jour in jours_logiques(depuis, jusqua):
        print(f"--- jour logique {jour}")
        # 2. une exécution du pipeline par jour ; la première qui échoue arrête le rattrapage
        if ____:
            print(f"rattrapage arrêté au {jour} : les jours suivants attendent")
            return 1
    print(f"rattrapage terminé : du {depuis} au {jusqua}")
    return 0

if __name__ == "__main__":
    # 3. le code de sortie du rattrapage, celui de sa première exécution en échec
    sys.exit(rattraper(sys.argv[1], sys.argv[2]))
```

Enregistrer, puis fermer le Bloc-notes.

Étape 2 — Le chargement d'un bloc, mis de côté · Tenir le pipeline

La preuve demande deux `RAW` à comparer : celle d'un chargement d'un bloc, et celle d'un rattrapage. La `RAW` se refait des sources par le pipeline : on peut l'effacer sans rien perdre. L'entrepôt, lui, est partagé : Analytics Engineering y bâtit ses vues et ses tables, à côté de la `RAW`. Data Engineering n'efface donc jamais le fichier, seulement ses propres tables, le `journal` et les `raw_…`. Le paquet de départ fournit `vider_tables_de.py`, qui fait ce tri et laisse le reste en place.

2.1 Charger le mois d'un bloc dans une `RAW` vide, puis mettre l'entrepôt de côté.

```
python ..\01_Data_Engineering\01_Brulerie_facile\Paquet_de_depart\vider_tables_de.py
REM Attendu : Tables de Data Engineering effacées : 9 (journal, raw_consentements, …, raw_tickets),
REM puis Objets gardés, ceux d'Analytics Engineering : 0, ou davantage si vous l'avez fait.
python 01_de\pipeline.py 2025-06-30
```


© 2026 iAlexMG. Tous droits réservés.

```
REM Attendu : bronze : 1527 tickets rangés, puis raw : 75 fichiers au journal.
copy plateforme\entrepot\brulerie.duckdb plateforme\entrepot\brulerie_bloc.duckdb
REM Attendu : 1 fichier(s) copié(s).
python ..\01_Data_Engineering\01_Brulerie_facile\Paquet_de_depart\vider_tables_de.py
REM Attendu : Tables de Data Engineering effacées : 9, comme au premier passage.
```

`brulerie_bloc.duckdb` est le témoin : le mois chargé d'une seule exécution. C'est une copie du fichier entier : les objets d'Analytics Engineering y sont aussi, s'il y en a, et l'étape 4 ne les regarde pas.

Étape 3 — Rattraper juin, jour par jour · Tenir le pipeline

3.1 Rattraper du 1er au 30 juin, dans la `RAW` vide. Comptez une quinzaine de secondes : trente exécutions, chacune avec son bronze refait.

```
python 01_de\rattraper.py 2025-06-01 2025-06-30
REM Attendu : trente fois --- jour logique, du 2025-06-01 au 2025-06-30, chacune suivie de
REM bronze : 1527 tickets rangés et de raw : N fichiers au journal ; puis
REM rattrapage terminé : du 2025-06-01 au 2025-06-30.
echo %ERRORLEVEL%
REM Attendu : 0.
```

Lire comment le journal grandit, jour après jour :

Jour logique	Fichiers au journal	Ce qui est arrivé ce jour-là
2025-06-01	7	l'export du 1er juin, ses tickets et ses lignes, et les cinq fichiers livrés en entier : le journal des lots, le catalogue et ses modifications, les membres et leurs consentements
2025-06-03	12	les exports du 2 et du 3, et la première courbe du torréfacteur
2025-06-11	30	autant qu'au cours 03, chargé jusqu'au 11
2025-06-13	37	les deux exports du 12, dont le repris, celui du 13 et sa courbe
2025-06-30	75	tout le mois

Chaque exécution n'a pris que ce qui était arrivé à son jour logique, et le journal a écarté tout ce qu'il connaissait déjà.

© 2026 iAlexMG. Tous droits réservés.

Étape 4 — Prouver que le rattrapage égale le chargement d'un bloc · Tenir le pipeline

Deux `RAW` égales ont les mêmes lignes, chacune autant de fois, et rien de plus. `EXCEPT ALL` rend les lignes d'une requête qui manquent à l'autre, exemplaire par exemplaire : dans les deux sens, il dit ce qui manque au rattrapage, puis ce qu'il a en trop. L'opérateur s'apprend au Cours 07 de la formation SQL avancé, `Theorie_07_SQL.md`, section 6, avec `UNION ALL`, qui met les lignes de deux requêtes bout à bout.

4.1 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Créer un notebook et le renommer « Cours 06 ». Dans sa première cellule, attacher les deux entrepôts en lecture seule, chacun sous son nom :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/brulerie.duckdb' AS rattrape (READ_ONLY);
ATTACH IF NOT EXISTS 'plateforme/entrepot/brulerie_bloc.duckdb' AS bloc (READ_ONLY);
-- Attendu : rattrape et bloc paraissent sous Attached databases.
```

4.2 Comparer les trois tables que Data Engineering lit, dans une cellule neuve.

```
-- 1. chaque table de la RAW, dans les deux sens : ce qui manque au rattrapage, puis ce qu'il a en trop
SELECT 'raw_tickets' AS table_raw,
       (SELECT COUNT(*) FROM (SELECT * FROM bloc.raw_tickets EXCEPT ALL SELECT * FROM rattrape.raw_tickets)) AS manquent,
       (SELECT COUNT(*) FROM (SELECT * FROM rattrape.raw_tickets EXCEPT ALL SELECT * FROM bloc.raw_tickets)) AS en_trop
UNION ALL
SELECT 'raw_lots',
       (SELECT COUNT(*) FROM (SELECT * FROM bloc.raw_lots EXCEPT ALL SELECT * FROM rattrape.raw_lots)),
       (SELECT COUNT(*) FROM (SELECT * FROM rattrape.raw_lots EXCEPT ALL SELECT * FROM bloc.raw_lots))
UNION ALL
-- 2. puis raw_courbes, de la même façon
SELECT 'raw_courbes',
       (SELECT COUNT(*) FROM (SELECT * FROM bloc.raw_courbes ____ SELECT * FROM rattrape.raw_courbes)),
       (SELECT COUNT(*) FROM (SELECT * FROM rattrape.raw_courbes ____ SELECT * FROM bloc.raw_courbes));
-- Attendu : 3 lignes, 0 et 0 pour chacune.
```

4.3 Comparer le journal. Les deux ont chargé les mêmes fichiers, aux mêmes empreintes ; l'instant du chargement, lui, diffère forcément, et ne se compare pas.

```
-- 1. les fichiers au journal, puis ceux qui ne sont que d'un côté, par leur nom et leur empreinte
SELECT (SELECT COUNT(*) FROM rattrape.journal) AS fichiers,
       (SELECT COUNT(*) FROM (SELECT fichier, empreinte FROM bloc.journal
                              EXCEPT ALL SELECT fichier, empreinte FROM rattrape.journal)) AS manquent,
       (SELECT COUNT(*) FROM (SELECT fichier, empreinte FROM rattrape.journal
                              EXCEPT ALL SELECT fichier, empreinte FROM bloc.journal)) AS en_trop;
-- Attendu : 75 fichiers, 0 et 0.
```


© 2026 iAlexMG. Tous droits réservés.

Le rattrapage égale le chargement d'un bloc : 1 585 lignes de tickets, 47 lots et 3 718 lectures de part et d'autre, et rien de plus. Les cinq autres tables de la `RAW` se comparent de la même façon, et le journal le dit déjà : les deux ont lu les mêmes fichiers, aux mêmes empreintes. Seul l'ordre du journal a changé : le rattrapage a noté le journal des lots dès le 1er juin, le bloc après tous les exports de la caisse. C'est ce qui permet de rattraper sans crainte : une période rattrapée jour par jour ne laisse aucune trace d'avoir été rattrapée.

Étape 5 — Le rattrapage qui s'arrête · Tenir le pipeline

Le notebook tient encore les deux entrepôts. Comme au cours 05, il suffit à faire échouer la `RAW`.

5.1 Rattraper les deux derniers jours pendant que le notebook tient l'entrepôt, dans la première fenêtre.

```
python 01_de\rattraper.py 2025-06-29 2025-06-30
REM Attendu : --- jour logique 2025-06-29, bronze : 1527 tickets rangés, raw : échec, IO Error…, puis
REM rattrapage arrêté au 2025-06-29 : les jours suivants attendent.
echo %ERRORLEVEL%
REM Attendu : 1.
```

Le 30 n'a pas été tenté : un rattrapage ne saute pas un jour en échec pour passer au suivant.

5.2 Rendre les entrepôts, puis relancer. Dans le notebook, une cellule neuve :

```
DETACH DATABASE IF EXISTS rattrape;
DETACH DATABASE IF EXISTS bloc;
```

Puis, dans la première fenêtre :

```
python 01_de\rattraper.py 2025-06-29 2025-06-30
REM Attendu : les deux jours, chacun avec raw : 75 fichiers au journal, puis
REM rattrapage terminé : du 2025-06-29 au 2025-06-30.
echo %ERRORLEVEL%
REM Attendu : 0.
```

Rien de neuf n'est entré : les deux jours étaient déjà chargés, et le journal l'a vu. Un rattrapage se relance après une panne sans rien doubler.

5.3 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

Étape 6 — Versionner · Tenir le pipeline

6.1 Effacer le témoin, qui a servi, puis valider le script.

© 2026 iAlexMG. Tous droits réservés.

```
del plateforme\entrepot\brulerie_bloc.duckdb
git -C 01_de add rattraper.py
git -C 01_de commit -m "Cours 06 : rattraper une période, jour logique par jour logique"
REM Attendu : 1 file changed.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

À retenir

- Le jour logique est la journée dont une exécution traite les livraisons, pas celle où elle tourne : rattrapé plus tard, le 12 juin charge encore ce qui était arrivé le 12.
- Un rattrapage lance le pipeline jour par jour, dans l'ordre, et s'arrête au premier échec ; relancé, il ne double rien, parce que le journal a tout vu.
- Rattraper un mois jour par jour égale le charger d'un bloc : on le prouve en comparant les deux `RAW` dans les deux sens.

© 2026 iAlexMG. Tous droits réservés.